ChatGPT

Executive Summary

Advanced “JOCKY”-style tools propose combining novel languages, automated polymorphism, fileless execution and kernel subversion to evade modern AV/EDR. This analysis examines research and real-world evidence for each component. We discuss the **threat model** and ethical/legal considerations of such dual-use technology. Next, we dissect the **technical architecture**, including language/compiler choices (LLVM IR vs custom), cross-platform toolchains (Windows/Ubuntu), and CI/CD-based build pipelines. We then survey **signature/heuristic evasion** methods: control-flow obfuscation, token/binary layout alteration, automated polymorphic packing, and CI/CD integration【19†L61-L69】【32†L100-L109】. We detail **fileless/in-memory techniques** (process hollowing, reflective DLL injection, direct syscalls, thread hijacking) and **living-off-the-land** abuse, with feasibility, required privileges, complexity, and detection signatures. We analyze **kernel-level subversion** via “Bring Your Own Vulnerable Driver” (BYOVD): how attackers find and load signed flawed drivers to disable EDR (e.g. the Qilin/Warlock ransomware chains)【34†L328-L332】【15†L230-L234】. Next, we cover **network/C2 management**: central controller using trusted cloud/CDN, domain fronting, and legitimate cloud APIs (e.g. Microsoft Graph) for stealthy communications【43†L18-L27】【47†L430-L442】. For each technique we discuss defensive countermeasures (e.g. tamper protection, driver blocklists, memory forensics, TLS inspection). We survey prior art (academic and industry), including recent papers and whitepapers, summarized in the accompanying table of publications. Finally, we list assumptions (privileges, jurisdictional compliance) underlying this analysis.

Threat Model and Legal/Ethical Considerations

- **Adversary Goals:** Run deep forensic or analysis scripts while avoiding detection by AV/EDR, or equivalently, an attacker aiming to execute code undetected. Goals include privilege escalation, persistence, and data exfiltration in a stealthy manner【34†L328-L332】【22†L586-L593】. We assume attackers have *user or administrator privileges* on target hosts, and initial network connectivity.
- **Knowledge & Capabilities:** The adversary has offline capability to compile and obfuscate code (e.g. in-house language compiler and CI/CD), and knowledge of AV/EDR detections (e.g. through YARA/Sigma rules, security blogs)【8†L336-L344】. They can leverage legitimate cloud infrastructure for C2 and possess or create custom vulnerable drivers.
- **Legal/Ethical:** This framework is **dual-use**. Its design to evade security tools can be used maliciously (likely illegal under cybercrime laws) or legitimately (e.g. law enforcement, internal forensics). We assume **no explicit jurisdiction**, so we caution that developing or using evasion tools may violate laws like computer misuse acts or driver-signing regulations. Ethical constraints are not specified, but analysts should ensure compliance with privacy and export laws. This report focuses on *analysis and defense*, not on instructing illicit use.
- **Assumptions:** We explicitly assume (1) attacker has appropriate privileges (often local admin) to load drivers or inject code; (2) network allows domain-fronted/cloud C2 (some providers now block domain fronting); (3) existing AV/EDR rely on known signatures, common compiler artifacts, user-mode API hooks, and kernel callbacks; (4) any new language (JOCKY) or IR can be integrated into standard build pipelines; (5) defense teams have some means (e.g. Sysmon, network logs) to detect anomalies.

Technical Architecture and Compiler/Frontend Choices

- **Language/IR Design:** A custom language (“JOCKY”) or novel IR can hinder static signature matching. For example, compiling in uncommon languages (e.g. Nim, Go, Rust) produces binaries with unique runtime stacks and metadata that evade patterns tuned for MSVC or GCC【3†L49-L58】【33†L100-L109】. Apostolopoulos *et al.* demonstrate that rewriting malware in less-used languages (Delphi, Python, Rust, Go) drastically reduced AV detections【30†L26-L30】【33†L88-L97】. Thus, a JOCKY frontend could output LLVM IR or a custom IR, enabling aggressive obfuscation passes. LLVM is attractive: it supports cross-platform backends (PE/ELF) and rich obfuscation (control-flow flattening, bogus flows, opaque predicates, etc.)【19†L61-L69】【33†L104-L112】. A custom IR could further deviate program tokenization. In either case, the IR should encode semantics while enabling transformations that confuse pattern-based scanners【19†L119-L128】【21†L61-L69】.
- **Cross-Platform Toolchain:** To target both Windows and Ubuntu, the toolchain might use LLVM/Clang (generating .exe or ELF), or separate compilers (e.g. LLVM for Windows, GCC for Linux). Using LLVM allows identical obfuscation passes on both platforms【19†L61-L69】. The compiler must produce position-independent code (for injection) and include support for native APIs on each OS. Custom runtime or interpreter components can be used (e.g. embedding a small VM) to avoid standard library artifacts. Build pipelines could leverage Docker or CI runners on both OSes to automate builds.
- **CI/CD-Driven Builds:** A continuous integration/continuous deployment (CI/CD) system automates obfuscation/packing on each build. For example, a GitLab/GitHub Actions pipeline can feed code through tools like LLVM-Obfuscator or custom shuffling scripts, then compile, sign (if needed), and distribute. Recent research (“AutoBypass”) suggests using knowledge bases and LLM agents in the pipeline to autonomously apply evasive transformations【8†L336-L344】【10†L102-L110】. At each commit, the CI could randomly select obfuscation passes (flattening, instruction substitution, encryption of constants) so that every binary instance has unique hashes and modified import tables【10†L105-L110】. This neutralizes signature/heuristic matching (anti-reputation) since no two builds look the same.

```
flowchart LR
  A[**Central Controller** (Cloud/CDN)] -->|Domain-Fronted C2| B[Client Agents
(Windows/Linux)]
  B --> C[**In-Memory Runner**]
  C --> D{Execution Techniques}
  D --> E[Process Hollowing / DL Injection]
  D --> F[Reflective DLL Injection]
  D --> G[Direct Syscalls & Unhooking]
  D --> H[Thread Execution Hijacking]
  D --> I[BYOVD Kernel Driver Load]
  I --> J[Terminate EDR Processes / Unhook Callbacks]
  C --> K[Collect Forensics / Logs]
  B --> L{Updates via CI/CD}
  L --> C
```

Figure: Proposed "JOCKY" architecture. A central interface uses trusted cloud (CDN) for C2, sending obfuscated scripts to client agents. Clients execute code in-memory using advanced techniques, optionally loading signed vulnerable drivers to disable EDR (BYOVD). Detection and logs flow back to management.

Signature and Heuristic Evasion Techniques

1. **Control-Flow Obfuscation:** Altering the program's control-flow graph (CFG) is fundamental. Techniques include *flattening* (adding dispatcher loops), *bogus control-flow injection* (fake jumps/loops), and *instruction substitution*【19†L61-L69】【21†L51-L60】. LLVM-based obfuscation passes (Obfuscator-LLVM) can split basic blocks or insert junk code so that function prologues/epilogues and API sequences deviate from known patterns. For instance, Ekin Böke's "Digital Camouflage" shows that even sophisticated LLM-based detectors fail when C code is compiled with LLVM obfuscation (flattening, bogus predicates)【19†L61-L69】. In practice, JOCKY's compiler frontend could randomly reorder blocks, insert opaque predicates, or utilize functional transformations like Continuation-Passing Style, following Wang *et al.* or Lan *et al.*【33†L152-L161】. **Feasibility:** Easy with LLVM passes; lightweight if limited to basic obfuscation, heavier with deep virtualization. **Privileges:** No special privileges needed to obfuscate; only at compile-time. **Complexity:** Moderate; standard tools exist, but custom passes may be needed for unique IR. **Detection:** Obfuscation defeats static signatures; dynamic analysis may still see same syscalls but in unusual order. Defenders need control-flow graph analysis or behavioral correlators. **Mitigation:** Use emulation-resistant scanning (extract semantics post-obfuscation), or rely on "whitelisting" via runtime allowlists (e.g. only allowing known good code structures). Tools like packer scanners (heavy packet emulation) can be used, but persistent obfuscation requires constant updates to detection logic.

2. **Token/Binary Layout Alteration:** Changing the binary's byte-level appearance (symbols, strings, section names) evades signature-based scans. This includes randomizing global symbols, packing variables at dynamic offsets, and encrypting string literals or critical constants. Polymorphic packers (like Shikata Ga Nai) generate a new mutation each build. As Apostolopoulos *et al.* note, even rewriting in a different compiler/toolchain can break signatures【30†L26-L30】. In CI/CD, each release of JOCKY scripts should incorporate unique encryption keys or variable encodings so the binary hash is different every time【10†L105-L110】. **Feasibility:** Straightforward; many open-source packers exist, or simply XOR/encrypt at compile time. **Complexity:** Low to moderate (just invoke an obfuscator in the pipeline). **Detection:** Modern AV may flag cryptor usage (entropy, known stub routines) if static; but heavy polymorphism forces reliance on heuristics/behaviour. **Mitigation:** Behavioral/heuristic engines (anomaly on unpack routines), integrity checks on executables (whitelisting known good checksums) and continuous threat intelligence for new packers.

3. **Automated Polymorphic Engines & CI/CD:** Rather than manual repacking, a JOCKY framework would integrate an automated polymorphic engine in its CI/CD pipeline【10†L105-L110】. As each build artifact moves from source to deploy, it passes through obfuscators, symbol randomizers, and variable-encryption routines. This mimics "auto-packing" with random seeds, so that entries points, import tables, and section names differ. For example, AutoBypass shows LLM-driven toolchains can produce hundreds of unique payloads tailored to evade specific EDR signatures【10†L105-L110】. **Feasibility:** High; simply scripted into build steps. **Complexity:** Implementation of the pipeline is moderate (or leverage existing build automation). **Detection:** Unique hashes frustrate reputation engines; defenders must shift to heuristic rules. **Mitigation:** Employ runtime behavioral analysis or ML models on execution patterns; monitor for abnormal post-unpacking actions.

4. **Living-off-the-Land (LOTL):** Instead of calling noisy WinAPI (e.g. VirtualAlloc, CreateProcess), JOCKY scripts can leverage native signed system utilities. For instance, use PowerShell or WMI to execute code in-memory【52†L7-L15】. Tools like *PowerSploit* and *LOLBins* can launch payloads. By avoiding custom binaries entirely, fileless attacks bypass static scanners and appear as normal system behavior【52†L23-L32】. **Feasibility:** Very high; abused by many attacks. **Privileges:** Often none beyond those for PowerShell/WMI (standard user usually fine). **Complexity:** Low; simply embed commands in script. **Detection:** Many EDRs monitor common LOLBins; signatureless. Defenders use AMSI or logging (Sysmon ScriptBlock)【52†L55-L64】. **Mitigation:** Enforce restrictions (AppLocker/WDAC) on scripting hosts, use Command-Line audit policies, and implement AMSI hooking with monitored outputs.

Fileless/In-Memory Execution Methods

1. **Process Hollowing:** The attacker creates a new process in suspended state (often a known benign EXE), unmaps its legitimate code, and replaces it with malicious shellcode or DLL before resuming the process. This "hollow" process then runs attacker code under the guise of the original process name. **Feasibility:** Mature technique; implemented in Cobalt Strike, Metasploit. **Privileges:** Requires at least what's needed to spawn a process and write its memory (standard user can hollow child processes of same user). **Complexity:** Medium; must handle PE relocation if injecting a full executable. **Detection Indicators:** Unusual DLL load into process, inconsistent process metadata (e.g. hashes). Monitoring tools like Sysmon 8 (CreateRemoteThread) can flag hollowing; specialized scanners check memory vs disk for mismatches. **Mitigations:** Use tools like `Process Explorer` / ETW to validate process image hashes; enforce code-signing and HVCI (Hypervisor Code Integrity) to prevent unsigned code execution even if injected.

2. **Reflective DLL Injection (Reflective Loading):** Here a DLL is loaded from memory rather than disk: the injector allocates memory, writes the DLL's bytes, and then jumps to its own loader stub to resolve imports and call `DllMain`. Tools like Metasploit's Reflective DLL Loader do this. **Feasibility:** High; standard payload. **Privileges:** Same as above (writeable to target process memory). **Complexity:** Medium; use existing reflective loaders. **Detection:** No file is written, so AV signatures on disk won't see it【22†L562-L565】. However, heuristics may catch the in-memory unpacking. One indicator is an EXE mapping itself as a DLL in memory. Security products can hook LoadLibrary calls or watch for PE headers in process memory. **Mitigations:** Enable "memory scanning" in EDR (detect abnormal PE structures in memory); lock down or monitor `VirtualAllocEx` / `WriteProcessMemory` calls; use managed code contexts (e.g. CLR DLLs injection is harder to hide).

3. **Direct System Calls & API Unhooking:** Modern EDRs typically hook user-mode API stubs (in ntdll.dll) to monitor actions. *Direct syscalls* bypass these hooks by invoking the kernel directly via `syscall` or `int 2e` instructions【38†L163-L171】. Alternatively, attackers may restore a clean copy of ntdll (or map a fresh copy) to remove hooks【38†L128-L137】. **Feasibility:** High for x86/x64 Windows; numerous public PoCs exist (Mimikatz *DirectSyscalls*). **Privileges:** No extra privileges beyond ordinary process rights. **Complexity:** Low to moderate; static tables of syscall numbers or dynamic retrieval needed. **Detection:** Endpoint can monitor the *absence* of expected API hooking (Gartner states the hard thing is noticing a hook being disabled)【38†L134-L142】. Palo Alto notes Cortex XDR can flag raw `syscall` instructions as suspicious if seen in user apps【38†L163-L171】. Defenders may capture Intel PT traces to see raw syscalls. **Mitigations:** Kernel-mode agents or ETW

can still catch syscalls. Utilize Supervisor Mode Execution Prevention (SMEP) to block user code in kernel, and use PatchGuard to detect modifications. Logging unknown code pages with executable content is also helpful.

4. **Thread Execution Hijacking:** Instead of creating a new thread, this technique takes an existing thread in another process and diverts it to malicious code. Steps: open a handle to a target process/thread, suspend the thread, allocate memory and write shellcode, use `GetThreadContext` / `SetThreadContext` to set the thread's instruction pointer to the shellcode, then resume the thread【41†L119-L128】. Unlike process hollowing, this repurposes an already-running thread. **Feasibility:** Proven in-the-wild (Lumma Stealer, ZLoader)【41†L134-L142】【41†L142-L148】. **Privileges:** Need rights to manipulate the target process/thread (usually possible if same user; admin needed if target is higher privilege). **Complexity:** Low; uses documented Win32 APIs. **Detection:** Look for `SuspendThread` / `SetThreadContext` calls to unusual processes. EDR may alert on context changes. Sysmon 8/9 (Remote thread injection alerts) and thread call stack anomalies (thread starts at non-module addresses) are indicators. **Mitigations:** Harden sensitive process (e.g. sign/whitelist critical services), employ ETW to log context changes, and monitor for threads resuming at non-standard entry points.

5. **Advanced Fileless Hooks:** Attackers may also hook system routines at runtime (e.g. inline patching of APIs), or use less common mechanisms like Windows User-Mode Callbacks, NTFS alternate data streams (ADSes), or IO port events. They may disable AMSI (Antimalware Scan Interface) by patching `amsi.dll` in memory or abusing privileged tokens (bypass UAC) to keep code ephemeral. Defenders counter by integrity-checking `amsi.dll` or monitoring unusual in-memory patches.

BYOVD and Kernel-Level Techniques

Bring Your Own Vulnerable Driver (BYOVD): Attackers load a *legitimate but vulnerable* signed kernel driver to jump to Ring 0 and disable security. In Windows 64-bit, unsigned drivers are blocked, but attackers simply *bring* signed ones (often outdated HW/drivers) that have exploitable IOCTLs. Cyrpto ransomware groups (Qilin, Cuba, Warlock) weaponized this: e.g. Reynolds ransomware bundled a vulnerable "NSecKrnl" driver (CVE-2025-68947) to kill EDR processes【22†L586-L593】; Druva reports that Qilin/Warlock use BYOVD to terminate *300+* security products before encryption【34†L309-L317】【34†L344-L353】.

- **Vulnerable Driver Discovery:** Analysts maintain catalogs (e.g. *LOLDrivers*【24†L100-L109】) of drivers with unsafe IOCTLs or design flaws (capcom.sys allows SMEP disable【24†L129-L133】; winio.sys allows arbitrary physical memory access【24†L135-L139】). Attackers scan vendor installers or public repos for signed drivers (often 2013–2016 era) lacking modern mitigations【24†L100-L109】. CVE databases are mined for such drivers.
- **Loading & Exploitation:** With admin (SeLoadDriverPrivilege) the attacker writes the driver file and calls `NtLoadDriver`. Windows (pre-HVCI) will load it despite vulnerabilities【24†L33-L42】【34†L328-L332】. Once loaded, they use its IOCTLs to arbitrary read/write kernel memory, bypass SMEP, or abuse insecure functionality【24†L125-L133】. At Ring 0, they can disable PatchGuard, zero EDR kernel callbacks (`PsSetCreateProcessNotifyRoutine` lists), and kill protected processes (bypassing PPL)【34†L328-L332】【34†L344-L353】. For example, after BYOVD load, Qilin's driver unhooks EDR callbacks and terminates AV services, leaving the endpoint "blind"【34†L366-L374】.

- **Feasibility:** High on Windows (as shown by real attacks). Requires only *local admin*. Drivers must be chosen carefully (signed and not yet blocklisted).
- **Complexity:** Moderate; need a small loader for the driver, and exploit code for its vulnerability (often trivial IOCTL calls). Also must handle 64-bit kernel constraints (some exploits fail on newer kernels/HVCI).
- **Detection Indicators:** Watch for unsigned or unusual signed driver loads (Sysmon DriverLoad event), especially if matching known vulnerabilities. Monitor kernel memory writes (ETW). Absence of EDR telemetry after loading a driver is a red flag【22†L619-L628】. Also, use of tools like *Tetradek* (BYOVD testkits) can be simulated to harden detection.
- **Mitigations:** Microsoft Vulnerable Driver Blocklist (WDAC) can prevent many known drivers【22†L681-L687】. Enabling Memory Integrity (HVCI) blocks loading drivers lacking HVCI-compatible signatures. Least-privilege (few admins), and kernel patch guard (detect corruptions) help. Implementing kernel-level intrusion detection (HIPs on kernel memory) and requiring approval for new driver loads is key【15†L245-L253】【22†L681-L689】.

Network and C2 Management

- **Central Management Interface:** JOCKY envisions a central controller that distributes tasks and receives results. This server can run in the cloud or CDN edge for scalability. Clients (endpoints) periodically poll or maintain an encrypted channel to the controller. All communications should **appear normal**: e.g. use HTTPS to known services (AWS, Azure, Google) and avoid custom ports. In practice, frameworks like Cobalt Strike have used HTTP(S)-based beaconing disguised as common web traffic.
- **Domain Fronting & Trusted Infrastructure:** To evade network filtering, attackers may use **domain fronting**【43†L18-L27】【45†L135-L143】. This means the TLS handshake's SNI field is a popular domain (e.g. `ajax.aspnetcdn.com`), while the encrypted HTTP `Host` header points to the real C2 (hosted on the same CDN). DeepStrike explains how this masks C2 as normal CDN traffic【43†L56-L65】. APT29's operations (e.g. using Tor's Meek transport) effectively did this【43†L125-L130】. While major clouds (AWS, GCP, Azure) largely disabled fronting by 2022【43†L45-L53】, creative variants still exist (e.g. abusing smaller CDNs or routing quirks).
- **Detection/Mitigation:** TLS inspection is required. Zscaler notes defenders should compare TLS SNI vs inner Host header, using a secure web gateway (SWG) with full SSL/TLS inspection【45†L169-L172】【45†L79-L87】. Blocking mismatched SNI/Host or enabling "block domain fronting" rules can close this gap.
- **Legitimate Cloud APIs:** An advanced C2 hides within legitimate cloud services. Recent malware has used **Microsoft Graph API** as a command channel: a payload authenticates via OAuth (client credentials) and polls user objects or Outlook messages for instructions【47†L273-L282】【47†L343-L350】. Since Graph over HTTPS to `graph.microsoft.com` looks normal, network monitors see nothing suspicious. Attackers also hide commands in comments of HTML on Cloudflare pages (e.g. RCREMARK) or in GitHub repos.
- **Examples:** The "GraphBrotli" backdoor (Go) embedded commands in user profile fields, decoding them (Brotli+Base91)【47†L273-L282】. "GraphRelook" issued commands via attacker-controlled Outlook emails【47†L338-L347】. Cloudflare-hosted servers returned HTML payloads with concealed instructions【47†L374-L383】.
- **Detection/Mitigation:** Because traffic is encrypted to cloud, defenders focus on **behavioral anomalies**. Monitor abnormal use of cloud APIs from endpoints (e.g. a workstation suddenly performing Microsoft Graph queries). Employ anomaly detection on identity logins or API calls. Use

DLP/SaaS governance tools to flag unusual patterns. Maintain threat intel lists of malicious OAuth apps or domains.

```
flowchart TB
    subgraph BYOVD Attack Chain
      A[Escalate to Local Admin] --> B[Drop Signed Vulnerable Driver];
      B --> C[Load Driver via NtLoadDriver];
      C --> D[Execute Arbitrary Kernel Code];
      D --> E[Disable AV/EDR Kernel Callbacks];
      D --> F[Terminate Security Processes];
      E --> G[Endpoint Blinded];
      F --> G;
      G --> H[Execute Payload (e.g. encrypt files)];
    end
```

Figure: Typical BYOVD attack chain. The attacker, now admin, installs a vulnerable signed driver into the kernel, exploits it to gain arbitrary ring-0 code execution, then disables EDR agents (unhooking or killing them), and finally runs the malicious payload completely undetected.

Detection and Defensive Countermeasures

- **Signature/Hash Evasion:** Since JOCKY's outputs change hash every build, defenders must move beyond static lists. Instead, use code behavior "whitelisting" (only allow known binaries/processes) or deep unpacking sandbox execution. Machine learning or LLM-based detection is also challenged by obfuscation【19†L61-L69】, so prioritize *runtime* heuristics.
- **In-Memory/Injection Detection:** Employ kernel callback monitoring: EDR products set `PsSetCreateProcessNotifyRoutine`, `PsSetCreateThreadNotifyRoutine` to see process/thread creation. Zero out callbacks (BYOVD) or bypass user-mode hooks (direct syscalls) avoid logging. To counter this, use ETW or Hypervisor introspection that can capture raw syscalls and thread events beyond user hooks. Sysmon and similar can log: `ProcessCreate`, `CreateRemoteThread`, `WriteProcessMemory`. Watch for threads resuming at strange addresses. Tools like Microsoft Sysmon 13/8 detect registry/WMI persistence and remote injection【52†L49-L58】.
- **BYOVD Mitigations:** As Cymulate and Druva advise, maintain an up-to-date driver blocklist (via WDAC) and apply OS patches to eliminate old vulnerabilities【15†L245-L253】. Use virtualization-based security (HVCI) to block unsigned/wrongly signed drivers from loading. Audit driver loads (Event ID 6) and kernel callback registrations. Alert if any non-Microsoft driver is loaded into critical devices. In high-security settings, disallow dynamic driver load by limiting SeLoadDriver privilege.
- **C2/Network Defenses:** Deploy network detection (NDR) alongside EDR【22†L649-L658】. Monitor for anomalous outbound connections (unusual domains, SSL to IPs without SNI). For domain-fronting and cloud abuse, enforce SSL inspection and SNI/host mismatch analysis【45†L169-L172】. Also implement "absence monitoring": EDR detection is often revealed when telemetry **stops**. Look for gaps in logs, missing heartbeats, or disabled agent services【22†L619-L628】. Use honeypots or decoy credentials in endpoints – true malicious injections may inadvertently trigger them.
- **Behavioral Analytics:** As the CyCraft report notes, static blocking fails when infrastructure is legitimate【47†L430-L442】. Focus on "living indicators" – e.g. if a process that normally never

spawns child processes suddenly does, or if a signed service unusually binds to an outbound address. AI-driven correlation engines (SOAR/XDR) can link network and process anomalies to catch "blind" attacks.

Prior Art and Research Summary

Title & Reference	Authors/Year	Venue/ Type	Key Findings	Relevance to JOCKY Framework
Coding Malware in Fancy Programming Languages for Fun and Profit 【30†L26-L30】 【33†L88-L97】	Apostolopoulos *et al.*, 2025	CODASPY (draft)	Malware in uncommon languages (Delphi, Python, Go, Rust) evades static AV; ported RATs showed much lower detection. Studied language-specific binaries and obf.	Demonstrates language choice drastically affects AV heuristics; supports JOCKY using a novel language/IR to bypass signatures.
"Digital Camouflage": The LLVM Challenge in LLM-Based Malware Detection 【19†L61-L69】	Ekin Böke, 2025	arXiv	Showed that LLM-based detectors (ChatGPT-4o, Gemini, Claude) often fail when code is compiler-obfuscated via LLVM (CF-flattening, bogus flows). Detection metrics (F1) dropped sharply after obfuscation.	Validates that LLVM-level obfuscation undermines even advanced detectors. Justifies using LLVM IR transformations in JOCKY to defeat analysis.

Title & Reference	Authors/Year	Venue/ Type	Key Findings	Relevance to JOCKY Framework
Mutate to Bypass: Autonomous Endpoint Evasion via Knowledge-Driven Multi-Agent Orchestration【10†L102-L110】	Yuan *et al.*, 2026	arXiv (CCS 2026)	Proposed AutoBypass: multi-agent LLMs with an evasion knowledge base. Automatically mutate shellcode loaders through CI/CD to evade EDR. Achieved ~90% evasion on WinDefender and others. Knowledge base (YARA/Sigma rules, blog data) improved evasion.	Illustrates automated CI/CD-driven polymorphism and closed-loop testing. JOCKY's CI/CD pipeline would adopt similar automated obfuscation and testing techniques.
Weaponizing Trust: How BYOVD Tactics Silently Bypass EDR Defenses 【34†L328-L332】【34†L344-L353】	Deshpande (Druva), 2026	Druva Whitepaper	Described Qilin/ Warlock ransomware using BYOVD as pre-encryption: load signed drivers at R0 to kill 300+ security processes. Outlined chain: DLL sideload → driver load → kernel code → AV termination.	Directly relevant BYOVD case study. Confirms feasibility of signed driver loads and kernel callback unhooking to blind EDR, exactly the kernel-level technique JOCKY might use (with a BYOVD component).

Title & Reference	Authors/Year	Venue/ Type	Key Findings	Relevance to JOCKY Framework
Understanding BYOVD Attacks and Mitigation Strategies【16†L19-L27】【16†L61-L69】	Freed, 2025	Halcyon (blog)	Survey of BYOVD adoption: 2019 RobbinHood (Gigabyte driver CVE-2018-19320), Lazarus/Cuba (killing AVs), BlackByte. Notes rising trend (23% increase in Q2 2024) and known BYOVD tools (Terminator). Discusses defenses (patching, whitelisting).	Provides real examples and statistics of BYOVD use, confirming it as an effective EDR bypass. Supports the BYOVD threat model for JOCKY and underscores need for driver hardening.
What is BYOVD? (TechZone)【24†L16-L25】【24†L125-L133】	Bitdefender Labs, 2024	Industry Blog	Explains BYOVD fundamentals: attackers "bring" signed drivers to escalate to R0. Describes escalation chain (admin → load driver → exploit). Gives examples (Capcom.sys, Winio). Notes attacker tactics (LOLDrivers, scanning installers).	Background on how BYOVD works and why signed status is trusted by OS. Validates strategy of using legitimate drivers. Informs JOCKY's BYOVD module design (scanning LOLDrivers repo).
Domain Fronting: Stealthy Traffic Masking Explained【43†L18-L27】【43†L125-L130】	Khalil, 2026	DeepStrike Blog	Survey of domain fronting: front domain in TLS SNI vs hidden Host header. Used by activists and APTs (APT29 via Tor/ meek) to hide C2. Explains why it evades normal filtering.	Details how CDN/ cloud fronting can hide malware C2. Relevant for JOCKY's network layer: suggests using fronting over Azure/AWS (if possible) and the risk of detection.

Title & Reference	Authors/Year	Venue/ Type	Key Findings	Relevance to JOCKY Framework
C2 Laundering via Microsoft Graph API【47†L430-L442】	Xue *et al.*, 2026	CyCraft (blog)	Case study of APT using cloud APIs: Malware (GraphBrotli/ GraphRelook) abused Microsoft Graph (user fields, Outlook messages) as C2. Concludes "Cloud Service Abuse" yields stealthy channels.	Demonstrates using legitimate cloud APIs for C2. Informs JOCKY's design to leverage Graph API or similar (Google Drive, AI APIs) for payload/ control, blending with normal enterprise traffic.
Fileless Malware: Why it Evades Antivirus and How to Stop it【52†L31-L40】【52†L69-L78】	Vectra AI, 2024	Industry Article	Defines fileless vs LOTL. Notes rise in campaigns (EggStreme, DEAD#VAX). Highlights memfd_create for in-memory execution on Linux (ShadowHS). Discusses detection strategies (memory forensics, XDR).	Provides context on living-off-land and memory-based threats across Windows/ Linux. Affirms JOCKY's use of memory-resident attacks (Process injection, in-memory payloads) and suggests layered defenses (memory scanning, behavioral XDR).

Unstated Assumptions

- **Operational Environment:** We assume JOCKY runs in a corporate or lab environment under the control of the attacker (or forensic analyst) with necessary privileges. Use on unmanaged systems or without authorization is outside scope and likely illegal.
- **Continuous Testing:** We assume availability of testbeds to verify that obfuscated builds indeed bypass EDR (as done in AutoBypass【10†L102-L110】).
- **No AI Code Generation:** JOCKY's polymorphism is hand-engineered or pipeline-driven; we do not rely on generative LLM producing malicious code (to avoid direct offense instructions). References to LLM are for defensive research (AutoBypass) only.
- **Cloud Services:** We assume cloud providers' policies (e.g. AWS, Azure) still permit some form of domain fronting or can be abused via misconfigurations (even if major fronting was curtailed, research suggests new variants exist【43†L45-L53】).

- **Legal Compliance:** All research is for academic/defensive analysis. Building or deploying such a tool for illicit purposes would violate laws (CFAA, DMCA, etc.). We do not detail exploit code, in compliance with usage guidelines.

This report synthesizes current research (2025–2026) on advanced evasions. It does not provide step-by-step attack code, but rather a high-level analysis for defenders and researchers.
